ESGI · Récupération de données & architectures data modernes

Brawl Stars

Data Mining Pipeline

🟤 Bronze → ⚪ Silver → 🟡 Gold → 📊 Visualisations

API Brawlify · Python · MySQL · Architecture Médaillon

- requests
- mysql-connector
- pandas
- matplotlib
- seaborn
- plotly

Contexte & Architecture

Pourquoi ce projet ?

Pourquoi ce projet ?
Brawl Stars possède beaucoup de données descriptives sur ses personnages:
nom, rareté, classe, star powers, gadgets.

L'objectif de ce projet est de transformer ces données issues d'une API publique en **pipeline data structuré**, afin de pouvoir alimenter automatiquement :

un **wiki dynamique**
un **catalogue de brawlers**
des **visualisations analytiques**

Objectif du projet :
Collecte → Stockage brut → Normalisation → Analyse → Restitution

L'API Brawlify

```
GET api.brawlify.com/v1/brawlers
```

- API REST publique, sans auth
- JSON : id, name, rarity{name}, class{name}, starPowers[], gadgets[]
- Timeout 30s + raise_for_status() → HTTP 2xx garanti
- 1 seul endpoint utilisé


```
{ "id": 16000000, "name": "Shelly",
  "rarity": {"name":
"Trophy Road"},
  "starPowers": [...],
  "gadgets": [...] }
```


Architecture Médaillon

BRONZE
JSON brut immuable

▼

SILVER
Normalisation relationnelle

▼

GOLD
Vues SQL analytiques

▼

VISU
PNG + HTML interactif

Couche Bronze — extract_bronze.py

Principe Bronze : on ne modifie jamais la donnée avant insertion. Table append-only, chaque appel = nouvelle ligne horodatée (UTC). Historique immuable.

extract_bronze.py

```
import json, requests, mysql.connector
from datetime import datetime, timezone

API_URL = "https://api.brawlify.com/v1/brawlers"

def extract_and_store_brawlers():
    response = requests.get(API_URL, timeout=30)
    response.raise_for_status()  # lève HTTPError si code != 2xx
    raw_payload = response.json()
    conn = mysql.connector.connect(
        host="localhost", user="root",
        database=
"brawl_stars_db", port=3306
    )
    cursor = conn.cursor()

    cursor.execute(
        "INSERT INTO bronze_raw
         (source, extraction_date, payload)
         VALUES (%s, %s, %s)",
        ("brawl_stars",
         datetime.
now(timezone.utc),
         json.dumps(raw_payload))
    )
    conn.
commit()
```

Table bronze_raw

- id → PK auto-increment
- source → 'brawl_stars'
- extraction_date → datetime UTC
- payload → TEXT (JSON brut)
- created_at → timestamp auto

Pourquoi immuable ?

- Rejouer le pipeline repart du JSON d'origine
- Audit trail de toutes les extractions
- Silver se reconstruit depuis Bronze

`json.dumps(raw_payload)` → JSON JAMAIS modifié avant insert

Couche Silver — transform_silver.py

Principe Silver : on lit le dernier payload Bronze (ORDER BY extraction_date DESC LIMIT 1), on parse le JSON, on éclate en 3 tables normalisées. Chaque INSERT est idempotent.

transform_silver.py

```
cursor.execute(
    "SELECT payload FROM bronze_raw
     ORDER BY extraction_date
DESC LIMIT 1")
payload = json.loads(cursor.fetchone()["payload"])
brawlers = payload.get("list", [])

for brawler in brawlers:
    brawler_id = brawler.get("id")
    rarity     = brawler.get("rarity", {}).get("name")
    b_class    = brawler.get("class",  {}).get("name")

    cursor.execute("""
        INSERT INTO brawler (id, name, rarity, class)
        VALUES (%s, %s, %s, %s)
        ON DUPLICATE KEY UPDATE
            name=VALUES(name), rarity=VALUES(rarity),
            class=VALUES(class)
    """, (brawler_id, name, rarity, b_class))

    for sp in brawler.get("starPowers", []):
        cursor.execute(
            "INSERT IGNORE INTO star_power
             (id, brawler_id, name, description)
             VALUES (%s,%s,%s,%s)",
            (sp["id"], brawler_id, sp["name"], sp["description"]))
```

Schéma Silver (3 tables)

- brawler(id PK, name, rarity, class)
- star_power(id PK, brawler_id FK, name, desc)
- gadget(id PK, brawler_id FK, name, desc)
- FK: ON DELETE CASCADE

Idempotence — pourquoi c'est clé

- ON DUPLICATE KEY UPDATE → upsert propre si brawler déjà en base
- INSERT IGNORE → skip si star_power déjà inséré
- Pipeline rejouable sans corrompre la base

JSON imbriqué → tables plates

- rarity.name → champ plat
- class.name → champ plat
- starPowers[] → table star_power
- gadgets[] → table gadget

Couche Gold — transform_gold.py

Principe Gold : vues SQL analytiques construites sur Silver. Pas de duplication — calculées à la volée. Toutes en CREATE OR REPLACE VIEW → idempotentes.

Exemples de vues Gold (SQL)

```sql
CREATE OR REPLACE VIEW gold_brawlers_by_class AS
SELECT class, COUNT(*) AS brawler_count,
       GROUP_CONCAT(name) AS brawlers
FROM brawler GROUP BY class;

CREATE OR REPLACE VIEW gold_brawler_statistics AS
SELECT b.id, b.name, b.rarity, b.class,
    COUNT(DISTINCT sp.id) AS star_power_count,
    COUNT(DISTINCT g.id)  AS gadget_count,
    COUNT(DISTINCT sp.id) + COUNT(DISTINCT g.id)
                     AS total_abilities
FROM brawler b
LEFT JOIN star_power sp ON sp.brawler_id = b.id
LEFT JOIN gadget g ON g.brawler_id  = b.id
GROUP BY b.id;

CREATE OR REPLACE VIEW gold_rarity_distribution AS
SELECT rarity,
    ROUND(COUNT(*) *
100.0 / (SELECT COUNT(*) FROM brawler), 2)

AS pct
FROM brawler GROUP BY rarity
ORDER BY FIELD(rarity,
  'Trophy Road','Rare','Super Rare',
  'Epic','Mythic','Legendary','Ultra Legendary');
```

Les 8 vues Gold produites

gold_brawlers_by_class	Nb / classe
gold_brawlers_by_rarity	Nb / rareté
gold_gadgets_per_brawler	Gadgets / brawler
gold_star_powers_per_brawler	Star powers / brawler
gold_brawler_statistics	KPIs / brawler
gold_dashboard_summary	Métriques globales
gold_class_distribution	Distribution classes %
gold_rarity_distribution	Distribution raretés %

Visualisations & Synthèse technique

Outputs — visualize_gold.py

brawlers_by_class.png — matplotlib + seaborn

Barres horizontales + camembert — répartition par classe

brawlers_by_rarity.png — matplotlib

Histogramme avec couleurs officielles du jeu (#FFD700 Legendary...)

abilities_analysis.png — matplotlib + seaborn

Grille 2×2 : Top 10, scatter star powers vs gadgets, distributions

heatmap_abilities.png — seaborn

Heatmap de la moyenne capacités par rareté × classe

interactive_dashboard.html — plotly

Dashboard HTML interactif : pie, barres, scatter avec tooltips

Ce que ce projet démontre

Architecture Médaillon

Bronze immuable → Silver normalisé → Gold analytique. Standard pro (Databricks, Azure).

Idempotence complète

ON DUPLICATE KEY UPDATE + INSERT IGNORE + CREATE OR REPLACE VIEW → rejouable à l'infini.

Modélisation relationnelle

MLD en 3NF, FK avec CASCADE, séparation entités (brawler / star_power / gadget).

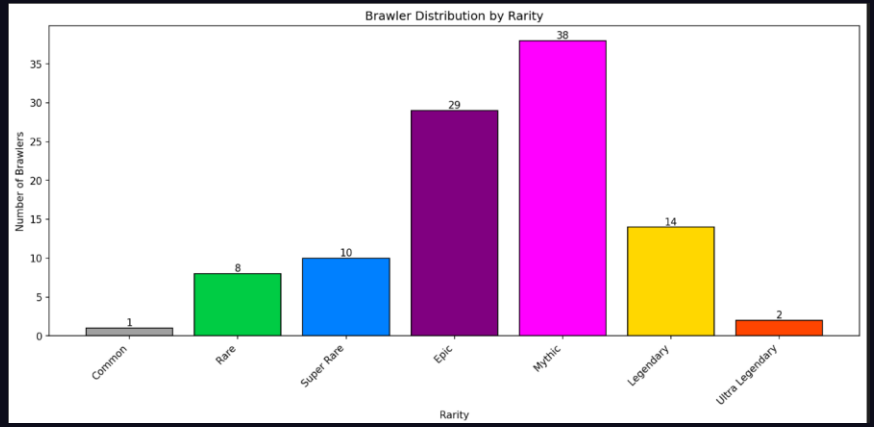
Pipeline Python bout-en-bout

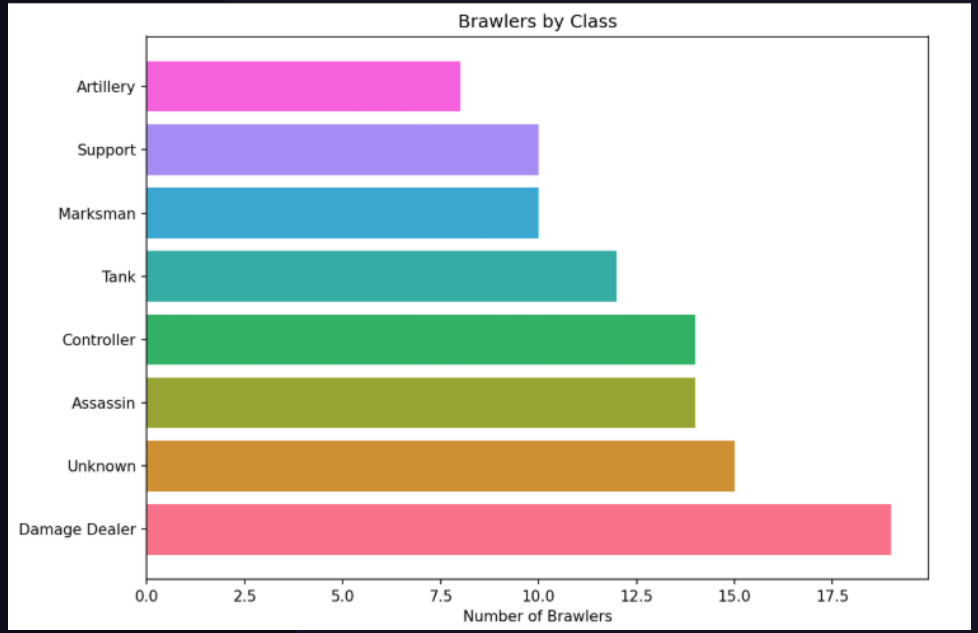
4 modules indépendants orchestrés par main.py, chacun exécutable via __main__.

Analyse & Visualisation

Requêtes sur vues Gold via pandas.read_sql(), 5 outputs dont dashboard Plotly interactif.

Data annalysables





Bilan technique

Bronze → Append-only

- Table bronze_raw = log immuable de toutes les extractions
- json.dumps() stocke le payload entier sans transformation
- On peut tout reconstruire à partir du Bronze
- raise_for_status() garantit la qualité de la source

Silver → Normalisation

- JSON aplati → 3 tables relationnelles typées
- ON DUPLICATE KEY UPDATE → upsert propre
- INSERT IGNORE → pas de doublons sur les capacités
- Les re-exécutions sont sans effet (idempotence)

Gold → Vues analytiques

- 8 vues SQL, toutes CREATE OR REPLACE VIEW
- COUNT(DISTINCT), GROUP_CONCAT, FIELD() tri custom
- Sous-requêtes scalaires pour les % de distribution
- Aucune donnée dupliquée — calcul à la volée

```
main.py  extract_and_store_brawlers()  →  transform_bronze_to_silver()  →  create_gold_views()  →
generate_all_visualizations()
```